Class vs Struct, and Relationships Between Classes in C#

A reference guide covering reference vs. value types, and how classes relate to one another beyond inheritance.

Part 1 — Class vs Struct

In C#, **class** and **struct** are both used to define custom types that group related data and behavior together. The fundamental difference between them is how the runtime treats their instances in memory: classes are **reference types**, and structs are **value types**. That single distinction drives almost every other difference listed below.

1.1 Core Comparison Table

Aspect	class	struct
Type category	Reference type	Value type
Memory location	Allocated on the heap	Allocated on the stack (or inline, if a field of another type)
Variable holds	A reference (pointer) to the object	The actual data itself
Assignment (a = b)	Copies the reference — both variables point to the same object	Copies the entire value — two independent copies exist
Default value	null	All fields zeroed / defaulted (never null, unless boxed or wrapped in Nullable<T>)
Inheritance	Supports inheritance from another class (single) and interfaces	Cannot inherit from a class or another struct; can implement interfaces
Can be a base class	Yes	No (structs are implicitly sealed)
Parameterless constructor	Implicit if none defined; suppressed once you add any constructor	Always implicitly available (all-zero state); C# 10+ also allows an explicit parameterless one
Passing to methods	Reference is passed (by default) — method can mutate the original object's data	A full copy is passed (by default) — method mutates only the copy, unless passed with 'ref'
Garbage collection	Managed and collected by the GC	No GC overhead — reclaimed automatically when it goes out of scope
Typical use case	Larger, complex objects with identity and mutable state (e.g. Car, Customer, FileStream)	Small, simple, immutable-ish data bundles (e.g. Point, Color, DateTime)

1.2 Code Example — Reference vs. Value Semantics

```
public class CarClass          // reference type
{
```

```
    public int Speed;
}

public struct PointStruct       // value type
{
    public int X;
    public int Y;
}

// --- class behavior ---
CarClass car1 = new CarClass { Speed = 100 };
CarClass car2 = car1;           // car2 now points to the SAME object as car1
car2.Speed = 200;
Console.WriteLine(car1.Speed);  // 200 -- car1 changed too!

// --- struct behavior ---
PointStruct p1 = new PointStruct { X = 1, Y = 2 };
PointStruct p2 = p1;            // p2 is an INDEPENDENT COPY of p1
p2.X = 99;
Console.WriteLine(p1.X);        // 1 -- p1 is unaffected
```

1.3 When to Choose Which

Use a **struct** when the type is small (rule of thumb: under ~16 bytes), logically represents a single value, is naturally immutable, and copying it is cheap and expected (e.g. coordinates, RGB colors, money amounts). Use a **class** for anything with identity that should be shared and mutated through references, anything that needs inheritance, or anything larger/more complex where copying the whole object on every assignment would be wasteful or semantically wrong.

Part 2 — Relationships Between Classes

Inheritance (“is-a”) is only one of several standard relationships classes can have with one another. The others describe how objects of different classes **collaborate** rather than share a type hierarchy. These come from object-oriented design / UML terminology and apply directly to C# code.

Relationship	Relationship type	Everyday phrase	Coupling strength
Inheritance	is-a	A Dog IS AN Animal	Strongest — shares implementation & type
Realization / Implementation	implements-a	A Car IMPLEMENTS IMovable	Strong — contract only, no shared code
Composition	has-a (owns)	A House HAS A Room (Room dies with House)	Strong — lifetime-bound ownership
Aggregation	has-a (uses)	A Department HAS Employees (Employees outlive it)	Medium — ownership without lifetime control
Association	uses-a / knows-a	A Student HAS A Teacher (both independent)	Loose — objects reference each other
Dependency	depends-on / uses temporarily	A Report USES A Logger inside one method	Weakest — momentary, not stored as a field

2.1 Inheritance (“is-a”)

A derived class inherits the members of a base class and can extend or override its behavior. This is a type relationship: the derived class genuinely IS a more specific version of the base class, and an instance of it can be used anywhere the base type is expected (polymorphism). C# supports single inheritance for classes only.

```
public class Animal
{
    public void Eat() => Console.WriteLine("Eating...");
}

public class Dog : Animal   // Dog IS AN Animal
{
    public void Bark() => Console.WriteLine("Woof!");
}
```

2.2 Realization / Implementation (“implements-a”)

A class fulfills a contract defined by an interface, providing concrete implementations for its members. Unlike inheritance, no implementation is shared from the interface (aside from optional default interface methods) — only the signatures/promises are shared. A class can realize many interfaces even though it can inherit from only one class.

```
public interface IMovable
{
    void Move();
}

public class Car : IMovable   // Car IMPLEMENTS IMovable
{
    public void Move() => Console.WriteLine("Car is moving.");
}
```

2.3 Composition (“has-a”, strong ownership)

One class owns another so completely that the owned object cannot meaningfully exist without it. The “part” is usually created inside the “whole” and shares its lifetime — when the owning object is destroyed, the owned object goes with it.

```
public class Engine
{
    public void Start() => Console.WriteLine("Engine starting...");
}

public class Car
{
    // Car creates and fully owns its Engine -- the Engine
    // has no independent existence outside a Car.
    private readonly Engine _engine = new Engine();

    public void Start() => _engine.Start();
}
```

2.4 Aggregation (“has-a”, weak ownership)

One class holds a reference to another, but the two have independent lifetimes: the “part” is typically created elsewhere and simply handed to (or shared with) the “whole,” and it can keep existing even after the whole is gone.

```
public class Employee
{
    public string Name { get; set; }
    public Employee(string name) => Name = name;
}

public class Department
{
    // Department does not create its Employees, and they can
    // exist (e.g. move to another department) independently.
    public List<Employee> Employees { get; } = new();

    public void AddEmployee(Employee e) => Employees.Add(e);
}
```

2.5 Association (“uses-a” / “knows-a”)

The most general relationship: two classes are aware of and can reference each other as peers, with no ownership or containment implied in either direction. It can be one-directional or bidirectional, and one-to-one, one-to-many, or many-to-many.

```
public class Teacher
{
    public string Name { get; set; }
}

public class Student
{
    // Student is associated with a Teacher, but neither owns
    // the other -- both can exist independently and a Teacher
    // may be associated with many Students at once.
    public Teacher AssignedTeacher { get; set; }
}
```

2.6 Dependency (“depends-on”, temporary use)

The weakest relationship: one class briefly uses another, usually as a method parameter, local variable, or return type — without storing a long-lived reference to it as a field. A change to the depended-on class can still affect the dependent class, but there's no ongoing structural link between instances.

```
public class Logger
{
    public void Log(string message) => Console.WriteLine(message);
}

public class ReportGenerator
{
    // ReportGenerator depends on Logger only for the duration
    // of this one method call -- it is not stored as a field.
    public void Generate(Logger logger)
```

```
    {
        logger.Log("Report generated.");
    }
}
```

2.7 Summary

Moving from strongest to weakest coupling: **Inheritance** and **Realization** define what a class fundamentally IS or must DO; **Composition** and **Aggregation** define what a class HAS (with differing lifetime control); **Association** defines what a class KNOWS about; and **Dependency** defines what a class momentarily USES. Recognizing which relationship fits a given design helps keep code loosely coupled — favoring association/aggregation/dependency over composition or inheritance wherever the tighter coupling isn't truly required is a common best practice (often summarized as “favor composition over inheritance”).